PRIDE Submission Workflow

2026-11-02

PRIDE upload

Instructions and templates for preparing a submission to PRIDE / ProteomeXchange.

See https://www.ebi.ac.uk/pride/markdownpage/submitdatapage for the PRIDE submission website, and Submission_Tutorial.pdf for the downloaded tutorial (v2.3.4) this template is based on.

Workflow

1. **Fill out PRIDE_submission_template.xlsx** with the project's metadata (see sheets below). Copy the template to a new file per project (e.g. `PRIDE_submission_<project>.xlsx`) rather than editing the template in place.
2. **List the files and compute checksums** with the scripts in `tools/` (see Repository contents): export the file lists, curate the `Files` sheet, then run the checksums. Checksumming the raw files can take up to a day, so consider starting it well before you intend to submit. Cached results mean an interrupted or repeated run only processes new or changed files.
3. **Generate the SDRF file** in the Excel template:

    1. Fill in one row per raw file in the `SDRF_Metadata` sheet (input sheet with ontology hints and guideline links). The `SDRF_tsv` sheet is the export view of these values. Column order matters: the `characteristics[...]` columns (including `characteristics[material type]`) come first, then `assay name` and `technology type`, then the `comment[...]` columns, with `comment[data file]` and the `factor value[...]` columns last. The template combines several SDRF templates (base, sample-metadata, human, ms-proteomics; optionally clinical-metadata and dia-acquisition), listed with links in the `Instructions` sheet. Trim the columns to those you use, and see the notes in rows 1 and 3 of `SDRF_Metadata` for how to fill each one.
    2. Export the **`SDRF_tsv`** sheet (not `SDRF_Metadata`) with `File > Save As > Text (Tab delimited) (*.txt)` (active sheet only), then rename the extension to `.sdrf.tsv`.
    3. Check that the `comment[data file]` values match the file names in the `Files` sheet and in `tools/checksum.txt` exactly (including extension and case). PRIDE cross-references the SDRF against the submitted files, so a mismatch will cause problems.
    4. Validate the file with sdrf-pipelines, the official SDRF validator:

    ```
    pip install sdrf-pipelines
    parse_sdrf validate-sdrf --sdrf_file your_file.sdrf.tsv --template human
    --template ms-proteomics
    ```

Pass one `--template` (`-t`) per template used in the `comment[sdrf template]` columns of the SDRF: the file is then validated against the union of all of them. Without `--template` only the default (`ms-proteomics`) is checked, so the human-specific columns would not be validated. For DIA data add `--template dia-acquisition`. `parse_sdrf list-templates` shows the available templates. Install `sdrf-pipelines[ontology]` to also validate ontology terms (or add `--skip-ontology` to skip that check). Fix any reported errors in the Excel sheet and re-export.

Use lesSDRF (CompOmics) only as a lookup tool for the correct formatting of column values; do not build the file there. See lesSDRF_manual.pdf for the app's documentation, and the SDRF specification for the online reference on column names and allowed values.

4. **Submit via the PX Submission Tool** (px-submission-tool-2.11.5/): use the `Dataset_Details`, `Files`, `File_Relationships`, and `Result_Metadata` sheets to populate the tool's fields, upload the listed files, and attach the generated SDRF file, to create the PRIDE/ProteomeXchange submission.

Template structure (`PRIDE_submission_template.xlsx`)

Sheet	Purpose
`Instructions`	Overview of the workbook and notes on controlled-vocabulary (CV) fields (species/NEWT, tissue/BTO, instrument/PSI-MS, modifications/PSI-MOD or UNIMOD, cell type/CL, disease/DOID).
`Dataset_Details`	One set of project-level metadata: submission type (Complete/Partial), result file format, title, keywords, description, sample/data processing protocols, MS experiment type, submitter and lab head contact details.
`Files`	One row per file to submit, with its `File type` (RAW, PEAK, SEARCH, QUANT, FASTA, SPECTRUM_LIBRARY, GEL, OTHER, or RESULT for Complete submissions) and location notes.
`File_Relationships`	Many-to-one join table linking each RESULT (or, for Partial submissions, SEARCH) file to the RAW/PEAK/FASTA/etc. files it depends on. Every RAW, PEAK, SEARCH, and FASTA file in `Files` should appear here at least once.
`Result_Metadata`	Per-RESULT-file annotation: species, tissue, instrument, experimental factor, modifica-

Sheet	Purpose
	tions/PTMs, cell type and disease (optional), quantification method.
`SDRF_Metadata`	Input sheet: one row per raw file/ sample, with SDRF columns (`source name`, `characteristics[...]`, `assay name`, `comment[data file]`, etc.), plus ontology hints and links to the SDRF guidelines.
`SDRF_tsv`	Export view of `SDRF_Metadata`. Save this sheet as tab-delimited text and rename to `.sdrf.tsv` to produce the SDRF file.
`Lists`	Dropdown value lists used for data validation elsewhere in the workbook (submission type, result file format, file type).

Partial vs. Complete submissions: Partial submissions don't provide RESULT files in mzTab/ mzIdentML form; use the SEARCH file (raw search engine output) as the "Result file" in `File_Relationships` instead, and leave `Result file format` blank in `Dataset_Details`.

Partial submission with MaxQuant — file type mapping:

File	File type (dropdown)
`proteinGroups.txt`, `peptides.txt`, `evidence.txt` (and other MaxQuant output tables)	RESULT
`mqpar.xml` (MaxQuant parameters / experiment design)	OTHER

(For Complete submissions the RESULT file must be mzTab or mzIdentML; `Result file format` is then set accordingly.)

Repository contents

- `PRIDE_submission_template.xlsx` — blank template, copy this per project.
- `tools/` — drop-in folder for the machine where the data lives. Copy the whole folder into the project's data directory and run the launchers by double-clicking; both scripts operate on the **parent** directory of the `tools` folder:
  - `Export-FileLists.bat` / `Export-FileLists.ps1` — **step 1:** scans recursively for proteomics files of interest (`.raw`, `.mzML`, MaxQuant output tables, `mqpar.xml`, `.fasta`, …) and writes one `.txt` per category into `tools/_filelists/`, grouped by folder (folder path, blank line, file names; two blank lines between folders). Copy names into the template's `Files` sheet and manually curate which files are actually submitted.

- `run_checksums.bat` / `make_checksums.py` — **step 2, after curating the `Files` sheet:** paste the curated file names into `tools/checksum_input.txt` (one per line; folder-path lines copied from the file lists tell it where the names below are) and run.
  - Finds each file, computes SHA-1 checksums in parallel (Python 3.6+ standard library only), and writes `tools/checksum.txt` (`file name<TAB>sha1`, the format PRIDE wants).
  - Never guesses: if a name is missing, matches more than one file, or is listed twice, it lists the problem lines and writes nothing.
  - Prints a progress line as each file finishes.
  - Saves results to `tools/checksum_cache.tsv`, so files unchanged since an earlier run (same path, size and modification time) are not checksummed again, and an interrupted run resumes.
  - Keep `make_checksums.py` compatible with Python 3.6: don't let a code formatter rewrite it for newer Python. (This is because 3.6 is what is available and I don't have admin rights…)

  Both scripts are self-contained: they only read the parent directory, and all output stays inside `tools/`.
- `px-submission-tool-2.11.5/` — the PX Submission Tool used to build and upload the submission.
- `README.pdf` — rendered copy of this README. To regenerate it, open `README.md` in Positron and run **Quarto: Render Document** from the command palette, or run `quarto render README.md` in a terminal.
- `Submission_Tutorial.pdf`, `lesSDRF_manual.pdf` — reference documentation (lookup only; the SDRF is built in the Excel template). Online SDRF reference: https://sdrf.quantms.org/index.html

Example `tools/checksum_input.txt`

Blocks can be pasted straight from the `_filelists/*.txt` files. A folder-path line applies to the file names below it; a name without a folder line (like `uniprot.fasta` here) is looked up anywhere in the project folder and must match exactly one file. Blank lines and `#` lines are ignored.

```
# Raw files
D:\Projects\Colonoids\Raw Data\Batch1

sample1.raw
sample2.raw


D:\Projects\Colonoids\Raw Data\Batch2

sample3.raw

# MaxQuant results (folder line needed: several runs
# have a proteinGroups.txt)
```

```
D:\Projects\Colonoids\MQ\run2\combined\txt

proteinGroups.txt
peptides.txt
evidence.txt

uniprot.fasta
```

Resulting `tools/checksum.txt` (file name, tab, SHA-1; in the same order as the input):

```
sample1.raw 764c16af46dd4f15edb05ecc5595b50cbe3714ea
sample2.raw 854d675a26dc0254a07b5725ba71242555863eb5
sample3.raw 4052402ba9118e4b5222aad52b7c87e949382b22
...
```